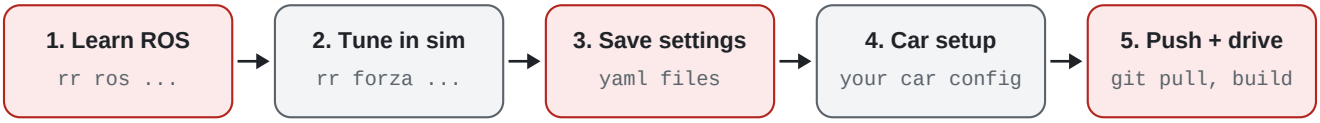
LEARNING GUIDE

From sim to real car

What ROS is, how ForzaETH works, how to tune it yourself, and how to push it onto your RoboRacer.

Companion to the RoboRacer Handbook · September 2026

The plan in one picture



Each chapter of this guide is one box. Do them in order; each one makes the next easier.

1	**ROS in plain words**	nodes, topics, parameters, launch files, builds
2	**How ForzaETH drives**	the pipeline from map to motor
3	**Tuning ForzaETH in the sim**	the knobs, what they do, how to know what to change
4	**A test plan**	which maps, which speeds, in what order
5	**Saving your settings**	which files, so tuning survives a restart
6	**Pushing to the real car**	what "push" means and the steps to ForzaETH on your car
7	**Cheat sheet**	every command from this guide

TIP Try every command in this guide yourself while reading. Nothing here can break anything: the worst case is rr forza stop, then start again.

1 ROS in plain words

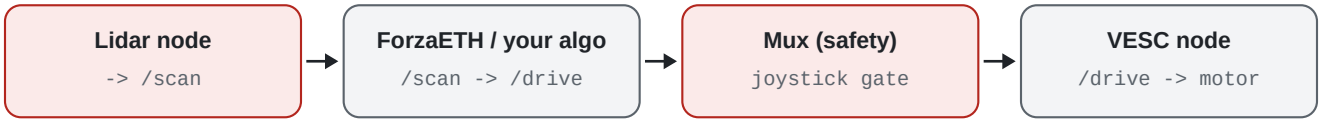
ROS (Robot Operating System) is not an operating system. It is a way to split a robot's software into small programs that talk to each other. Your car, the simulator and ForzaETH all use ROS 2 (version "Humble").

THE IDEA Every piece of the robot is a separate program. They never call each other directly; they publish and subscribe to named streams of data. Swap one program for another that uses the same streams and nothing else notices. That is why the sim and the real car can run the same code.

The five words you need

Word	What it is	On your car
Node	One running program with one job.	lidar driver, motor driver, your algorithm, ForzaETH's controller
Topic	A named stream of messages. Anyone can publish, anyone can listen.	`/scan` (lidar), `/drive` (speed + steering), `/odom` (where the car thinks it is)
Message	The data format on a topic.	`/scan` carries a LaserScan: about 1080 distances, 40 times a second
Parameter	A setting inside a node you can read or change while it runs.	`m_l1` on the controller, `global_limit` on the sector tuner
Launch file	A recipe that starts many nodes at once with their settings.	`time_trials_launch.xml` starts the controller, state machine, planner...

How the car's programs talk



The mux only lets /drive through while you hold the joystick's autonomous button. Let go = stop.

Look around yourself (start rr sim first; these are read-only and safe)

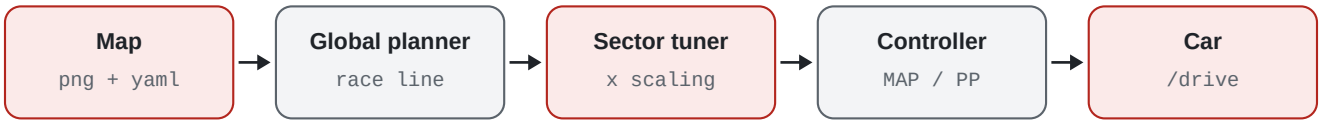
Command	What you will see
`rr ros node list`	Every running program (node).
`rr topics`	Every data stream (topic).
`rr echo /drive`	The speed and steering commands, live. Ctrl-C to stop.
`rr ros topic hz /scan`	How many lidar scans per second arrive.
`rr ros topic info /drive`	Who publishes and who subscribes to a topic.
`rr forza params controller`	Every setting of ForzaETH's controller (with rr forza sim running).

Workspaces and building

- Code lives in a **workspace**: a folder with `src/` (your code) and, after building, `build/` and `install/`.
- `colcon build` compiles every **package** in `src/` into `install/`. A package is one folder with a `package.xml` (ForzaETH has 28).
- `source install/setup.bash` tells your terminal where the built packages are. rr does this for you.
- Settings files (yaml) are linked, not copied, so editing them needs a restart but no rebuild. Adding new files needs a rebuild.

2 How ForzaETH drives

ForzaETH is ETH Zurich's full race stack. Unlike follow_the_gap, it plans the whole lap in advance and then follows that plan precisely.



Offline (once per track): map and planner. Live (every 25 ms): sector tuner, controller, car.

Piece	What it does	Where you see it
Global planner	Reads the track image, finds the centerline, then computes the fastest line (the "minimum curvature" line) and a speed for every point, based on how hard the car can corner and brake.	`rr forza addmap` runs it. Red and yellow dots in RViz.
Sector tuner	Multiplies the planned speeds by a factor per sector (piece of the lap). This is your main speed knob.	`rr forza speed`, `rr forza sector`
State machine	Decides the mode: follow the race line, trail a slower car, or overtake.	rr forza logs, window "drive"
Controller (MAP)	Looks a distance ahead on the race line and steers toward that point. The distance grows with speed.	Settings m_l1, q_l1, t_clip_min, t_clip_max
Lap analyser	Times every lap.	`rr forza laps`

> **TIP** The race line assumes perfect grip. Running near 1.0 (100%) can work in the sim; on the real car, start at 0.3 and earn your way up.

3 Tuning ForzaETH in the sim

Tuning is a loop: measure, change one thing, measure again, write it down.

1. Start: `rr forza sim Comp1`, then `rr forza race`. Open the browser view.
2. Measure: watch 3 to 5 laps and run `rr forza laps`. Note the lap time and anything bad (wall touches, wobbling, cutting corners). You do not need more laps than that per setting.
3. Change **one** setting with the commands below.
4. Measure again. Better? Keep it. Worse? Put it back.
5. Write it down (the log in chapter 4). When happy, save it to the files in chapter 5.

The knobs, in the order to learn them

Knob	Command	What it does	Start at
Overall speed	`rr forza speed 0.6`	Fraction of the race line's speed for the whole lap.	0.5 (sim), 0.3 (car)
One sector	`rr forza sector 1 0.4`	Slow down or speed up just one part of the lap (needs sectors, chapter 5).	same as overall
Lookahead gain	`rr forza param controller m_l1 0.35`	How much further ahead it aims per m/s of speed. Bigger = smoother but cuts corners.	0.3
Lookahead base	`rr forza param controller q_l1 0.2`	Aim distance at zero speed. Bigger = calmer at low speed.	0.15
Lookahead limits	`rr forza param controller t_clip_min 0.8`	Never aim closer than min or further than max (meters).	1 and 5
Steer anticipation	`rr forza param controller speed_lookahead_for_steer 0.1`	Steers a little earlier to make up for delay. Faster laps, more corner cutting. Max 0.2.	0

The controller aims at a point this far ahead: **distance = q_l1 + speed x m_l1**, kept between t_clip_min and t_clip_max. Almost every steering problem is this distance being too short or too long.

How to know what to change

You see	Why	Try
Clean laps, nothing scary	You have margin.	rr forza speed +0.05, repeat.
Touches the wall in one specific corner	That corner is too fast for the line.	Split that corner into its own sector and lower only it.
Swings side to side (wobble) on straights	Aiming too close: over-correcting.	Raise m_l1 or q_l1 a little.
Cuts the inside of corners, clips inner wall	Aiming too far ahead.	Lower m_l1, or lower speed_lookahead_for_steer.
Runs wide at corner exit	Too fast in the corner.	Lower that sector, or lower overall speed.
Slow but perfectly smooth	Leaving time on the table.	Speed up the straight sectors only.
Lap times jump around a lot	The car is near its limit somewhere.	Back off 0.05 and find the worst corner.
Crashes right after the start	Start pose or wrong map.	rr forza stop, check the map name, start again.

HEADS UP Change one thing at a time. If you change speed and lookahead together and it gets worse, you will not know which one did it.

4 A test plan

Go from easy to hard. Move on once the current stage is clean for 5 laps in a row.

Stage	Maps	Goal	Speeds to try
A. Known good	glc_ot_ez (ETH's real track)	Learn the commands; see what clean laps look like.	0.5, 0.6, 0.7, 0.8
B. Short and tight	Comp6 (27 m), Comp4 (28 m), Comp8 (31 m), Comp1 (34 m)	Hairpins and choke points: learn lookahead tuning.	0.5 up to the first touch, then back 0.05
C. Longer, faster	Comp7 (40 m), Comp5 (45 m), Comp3 (46 m); race lines hit 9-11 m/s	High speed into tight corners: learn sector tuning.	0.5 up; add sectors for the hard corners
D. New tracks	rr track new, then rr forza addmap CompN	Does your tuning work on a track it has never seen?	Start from your best settings
E. Racing	glc_ot_ez, rr forza h2h	Trailing and overtaking.	Your best time-trial settings

Keep a log

A simple table is enough. Keep it in a notes file, for example roboracer/notes/tuning.md:

```
map    speed  m_l1  q_l1  lookahead_steer  best lap  touches  notes
Comp1  0.50   0.30  0.15  0.0              7.21      0        baseline
Comp1  0.70   0.30  0.15  0.0              5.20      0        clean
Comp1  0.80   0.30  0.15  0.0              ?         ?        ...
```

The first two rows are real: that is what the setup check measured on Comp1.

Making a new track for ForzaETH

```
rr track new 42            # makes Comp42, a competition-style track
rr forza addmap Comp42     # computes its race line (a few seconds)
rr forza sim Comp42        # then: rr forza race
```

5 Saving your settings

Live changes (rr forza speed, rr forza param) disappear when the sim restarts. To keep them, write them into the files ForzaETH reads at start-up.

All paths below are inside `~/Documents/Projects/robotics/roboracer/forzaeth/ws/src/race_stack/stack_master/`. Edit them on your Mac with any text editor, then restart the sim (no rebuild needed).

What	File	Keys
Speed per track	maps/<map>/speed_scaling.yaml	global_limit, n_sectors, Sector0.scaling ...
Controller (sim)	config/SIM/l1_params.yaml	m_l1, q_l1, t_clip_min, t_clip_max, speed_lookahead_for_steer
Controller (your car)	config/<YOURCAR>/l1_params.yaml	same keys, tuned separately for the real car
Overtaking zones	maps/<map>/ot_sectors.yaml	Overtaking_sectorN.ot_flag: true where passing is allowed
Race line safety margin	config/global_planner/global_planner_params.yaml	safety_width (m): bigger keeps the line further from walls; then rr forza addmap again

Splitting a lap into sectors

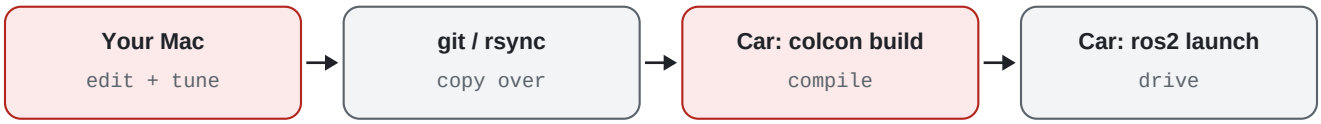
The race line is a list of numbered points (waypoints), about 200 on a Comp track. A sector is a range of them. In RViz the sector markers show where each sector starts. Example: three sectors on Comp1 (waypoints 0 to 181), slowing only a hairpin in the middle:

```
sector_tuner:
  ros__parameters:
    global_limit: 0.8
    n_sectors: 3
    Sector0: {start: 0,   end: 70,  scaling: 0.8,  only_FTG: false, no_FTG: false}
    Sector1: {start: 71,  end: 110, scaling: 0.55, only_FTG: false, no_FTG: false}  # hairpin
    Sector2: {start: 111, end: 181, scaling: 0.8,  only_FTG: false, no_FTG: false}
```

Sectors must cover every waypoint with no gaps, and global_limit caps every sector. After editing, restart the sim; then `rr forza sector 1 0.5` tunes that sector live.

6 Pushing to the real car

"Pushing" means three things: copy the code and settings onto the car, build them there, then launch them. The car is just a small Linux computer.



Route 1: your own algorithms (ready now)

rr already does this. `rr car-push` copies racer_algos with rsync and builds it on the car; `rr car-run follow_the_gap` launches it. The car just needs its IP in car.env. Do this first: it proves the Wi-Fi link, SSH, building on the car and the joystick safety gate all work, with simple code.

Before either route: this car needs laser odometry

Checked on the car 2026-09-22: the VESC connects and reports firmware 6.2, but publishes no sensor data with the driver installed. So there is no wheel odometry, no odom to base_link transform, and SLAM drops every lidar scan (mapping produces nothing). Two of the three causes are fixed; this one is not, and it does not affect plain driving, which needs only the lidar.

- `rr car-odom-setup` once: installs rf2o, which works out the car's motion by comparing lidar scans, plus the fusion that blends it with the VESC IMU (the IMU works; only the wheel packets are missing). It does not touch the VESC or its firmware.
- `rr car-odom` before mapping, left running in its own tab. It publishes /odometry/filtered and the odom to base_link transform that SLAM needs.
- `rr car-check` tells you in one screen which piece is missing if mapping stays empty.
- The other option, not taken because Ben does not want to touch the VESC: update the car's vesc_driver to a version that understands firmware 6.x, or put the VESC back on 5.x.

Route 2: ForzaETH on the car (the real goal)

ForzaETH has to live on the car, because the car drives even when your Mac is not connected. The steps:

1. **Find out what is on the car.** SSH in (rr car) and check: computer type (Jetson: `cat /etc/nv_tegra_release`), Ubuntu version (`lsb_release -a`), ROS version (`ls /opt/ros`), and the lidar model. ForzaETH's ROS 2 version needs ROS 2 Humble (Ubuntu 22.04), or its Docker image (they have a "jet" target for Jetsons).
2. **Get the code there.** Best: make your own copy (fork) of ForzaETH/race_stack on GitHub, commit your tuned yaml files to it from the Mac, and `git clone` / `git pull` it on the car. Then every "push" is `git push` on the Mac and `git pull` on the car.
3. **Install and build on the car**, following their install guide (Docker "jet" target on a Jetson, or the manual steps). Same idea as your Mac: dependencies, rosdep, colcon build.

4 **Make your car's config.** Copy `config/NUC2` to `config/YOURCAR`. In `vesc.yaml` set `speed_to_erpm_gain` (motor speed per m/s) and `steering_angle_to_servo_offset` (the value where the wheels point straight). Measure these on your car: wrong values mean it drives at the wrong speed or pulls to one side. In `config/sensors.yaml` set your lidar's IP or serial port.

5 **Map your track.** `ros2 launch stack_master mapping_launch.xml map_name:=mytrack racecar_version:=YOURCAR`. Drive one slow lap with the joystick; it builds the map and computes the race line, like rr forza addmap does in the sim.

6 **Start the base system**: `ros2 launch stack_master base_system_launch.xml map_name:=mytrack sim:=false racecar_version:=YOURCAR`.

7 **Time trials, slowly**: `ros2 launch stack_master time_trials_launch.xml racecar_version:=YOURCAR LU_table:=NUC2_hangar_pacejka ctrl_algo:=MAP`, with global_limit 0.3 and a hand on the joystick. The steering lookup table (LU_table) describes how your tires grip; borrow ETH's at first, and later measure your own with their system identification tools.

8 Tune exactly like in the sim, starting slow. Save settings to config/YOURCAR and maps/mytrack, commit, push.

> **HEADS UP** On the real car: wheels off the ground for the first run of anything new, global_limit 0.3 to start, a hand on the joystick, and nobody on the track.

What to ask your friend before starting

- The car's IP on the Wi-Fi, and the password for swat-roboracer-2026.
- Which computer it has (Jetson model?) and which Ubuntu and ROS version.
- Which lidar (Hokuyo UST-10LX, UST-30LX, other?) and how it connects (Ethernet IP or USB).
- Whether the VESC (motor controller) settings are already calibrated, and where they are saved.
- Which joystick button enables autonomous driving.

7 Cheat sheet

Everything in this guide, in one place.

Command	What it does
`rr forza sim [map]`	Start the ForzaETH sim (browser: http://localhost:6081).
`rr forza race`	Time trials on the race line (MAP controller). rr forza race PP = pure pursuit.
`rr forza h2h`	Head-to-head racing with overtaking.
`rr forza laps`	Lap times so far.
`rr forza speed 0.6`	Whole-lap speed, as a fraction of the race line's speed.
`rr forza sector 1 0.4`	Speed for one sector.
`rr forza param controller m_l1 0.35`	Change any setting live.
`rr forza params [node]`	List a node's settings (default: controller).
`rr forza maps`	Maps ForzaETH can drive.
`rr track new [seed]`	New competition-style track, named Comp<seed>.
`rr forza addmap Comp42`	Give a track to ForzaETH (computes its race line).
`rr forza logs`	Live logs of every ForzaETH node (Ctrl-b n = next window, Ctrl-b d = leave).
`rr forza stop`	Stop the ForzaETH sim.
`rr ros node list / rr topics / rr echo /drive`	Look around ROS (in the gym sim).
`rr car / rr car-push / rr car-run`	Real car: log in, send your algorithms, run one.
`rr down`	Turn everything off when done.